

부산 도시철도 역간 이동 경로 탐색 시스템

항목	내용
과목명	자료구조
프로젝트 제목	부산 도시철도 역간 이동 경로 탐색 시스템 (busan-metro-navigator)
이름	박해원
학번	20253279
제출일	2026년 6월 11일

1. 프로젝트 개요 및 실행 결과

1-1. 개요

부산 도시철도 전 노선(1~4호선, 부산김해경전철, 동해선)을 **인접 리스트** 그래프로 표현하고, 같은 출발·도착 쌍에 대해 **0-1 BFS(최소 환승 경로)** 와 **다익스트라(최단 시간 경로)** 두 알고리즘을 동시에 실행하여 결과를 비교 출력하는 프로그램이다.

```

과제/
├── build_graph.py      # CSV → metro_data.py 생성기 (+ 데이터 검증)
├── src/
│   ├── busan_subway.py # 메인 구현 (그래프 탐색 + 출력/입력)
│   └── metro_data.py   # 인접 리스트 데이터 (자동 생성)
├── data/
│   └── busan_metro.csv # 단일 진실 공급원 (line, order, station,
transfer)
└── docs/               # 계획서 및 SDD (spec_0, spec_1)

```

실행 방법: `cd src && python3 busan_subway.py` — 테스트 케이스 7건의 결과가 먼저 출력되고, 이어서 출발역/도착역을 직접 입력하는 대화형 모드로 진입한다(q 종료).

제약 준수: 외부 라이브러리 미사용 — `heapq`, `collections`, `dataclasses`, `csv` 등 표준 라이브러리만 사용. Python 3.8 이상.

1-2. 코드 및 코드 설명

코드 원본은 압축 파일에 동봉하였으며, **코드 설명은 소스 내 주석(모듈 `docstring`, 클래스·메서드 `docstring`)으로 대체한다.** 각 파일 상단에 데이터 형식·환승 모델·클래스 구성이, 각 메서드에 상태 정의와 동작 원리가 주석으로 명시되어 있다.

1-3. 실행 결과 (결과물)

실행 화면 캡처 이미지는 별도 파일로 첨부한다. 캡처에는 다음이 포함된다.

1. 테스트 케이스 7건의 비교 출력 — 출발/도착별로 [BFS] 최소 환승 경로와 [다익스트라] 최단 시간 경로를 나란히 출력
2. 대화형 입력 모드 — 정상 탐색, 존재하지 않는 역 입력 시 안내, 동명이역(좌천) 입력 시 노선 구분 안내

대표 결과 예시 (텍스트 발췌):

[BFS] 최소 환승 경로

경로 (24개역): 남포 → 중앙 → ... → 범어사 → 노포

환승: 0회 / 소요: 약 46분

2. AI Agent 활용 내역 — Prompt 원문, 출력 요약, 본인 수정 사항, 반영 여부

2-1. 활용 방식 개요: 스펙 주도(Spec-Driven) 워크플로

본 과제에서 AI Agent에 대한 프롬프팅은 일회성 지시문이 아니라 **버전 관리되는 스펙 문서(SDD)**를 입력으로 주는 방식으로 수행했다. 즉 "프롬프트 원문 = SDD 문서"이며, 구현 단계에서 AI에게 자연어로 길게 지시하는 대신 스펙을 먼저 확정하고 AI는 스펙을 실행하는 역할로 한정했다.

기획(계획서) → SDD 작성 → AI 구현 → 사람 검토(AI 보조) → SDD v1 (수정 스펙) → AI 수정 실행 → 검증 게이트 → 제출

산출물	위치	역할
계획서	docs/busan_subway_plan.md	자료구조 선정 근거 (인접 리스트 vs 행렬 비교)
spec_0	docs/sdd/busan_subway_spec_0.md	초기 구현의 프롬프트 원문
spec_1	docs/sdd/busan_subway_spec_1.md	데이터 오류 수정의 프롬프트 원문

2-2. 단계별 프롬프트 원문과 AI 출력 요약

단계 1a — 기획 및 스펙 작성 (대화형 세션)

프롬프트 방식: 대화형으로 계획을 공동 수립했다. 주요 결정 과정의 프롬프트(발체):

"그럼 저 3개의 방법 중 인접 리스트로 선택했다는 것도 과제 계획서에 포함시켜주면 좋을 것 같아"

AI 출력 요약: 과제 계획서(`busan_subway_plan.md` — 인접 행렬/리스트/엣지 리스트 3방식 비교와 선택 근거 포함), SDD(`busan_subway_spec_0.md` — Goals/Non-Goals, 데이터 형식, 함수 시그니처, 출력 형식, 테스트 케이스, 외부 라이브러리 금지 제약), 그리고 부산 도시철도 6개 노선의 역 데이터 CSV를 산출했다. 즉 **v0 데이터(CSV)는 AI가 생성한 것이다**. 이 세션에서 이미 일부 데이터 결함(괘법르네시떼 에지 비대칭, 동해선 신해운대 누락)이 발견되었으나 당시 우선순위에서 미뤄두었다 — 이 결함들은 이후 단계 2의 전수 리뷰에서 다른 오류들과 함께 다시 확인되었다.

단계 1b — 초기 구현 (Claude Code)

프롬프트 원문(전문): 스펙 문서가 컨텍스트로 주어진 상태에서, Claude Code에 입력한 자연어 프롬프트는 다음 2건이 사실상 전부다.

"근데 좀 객체 지향적으로 코드를 작성해줄 수 있어? class 함수를 사용해서"

"좋아. 그럼 나 지금 CSV로 데이터가 정해져 있잖아. 근데 나는 인접 리스트로 데이터로 나타내고 싶어. Python이면 좋겠어"

프롬프트가 짧을 수 있었던 이유는 요구사항이 이미 스펙 문서에 명시되어 있었기 때문이다. 즉 상세 지시는 문서가 담당하고, 대화 프롬프트는 방향 전환(객체지향 전환, 데이터 표현 방식 지정)에만 사용했다.

AI 출력 요약:

- `RouteResult` (값 객체) / `BusanMetro` (그래프+알고리즘) / `MetroApp` (입출력) 3계층 구현
- 최소 환승: deque 기반 0-1 BFS / 최단 시간: heapq 기반 다익스트라
- 환승역을 단일 노드로 병합하고 에지에 노선 라벨을 부여, 탐색 중 노선 변경 시 환승 3분 가산하는 모델
- CSV를 인접 리스트로 변환한 `metro_data.py` (역간 2분 균일 가중치)

단계 2 — 코드 리뷰 요청 (사람 + AI 교차 검토)

프롬프트 원문(발체):

"자 이제 자료구조 과제야. 한번 봐줄래? 일단 우리가 채택한 그래프랑 다익스트라는 다행히 마지막 수업 시간에 배워서 그대로 해도 좋을 것 같아."

AI 출력 요약: 알고리즘·OOP 구조는 유지 가능 판정. 단, 데이터에 실제 노선과 다른 체계적 오류 6건 발견:

ID	내용
C1	1호선: 동래 위치 오류, 4호선 전용역 5개(수안·낙민·총렬사·명장·서동) 잘못 삽입
C2	2호선: 광안 누락, 망미·재송·센텀 잘못 삽입, 센텀시티 누락
C3	3호선: 강서구청 ↔ 체육공원 순서 오류
C4	4호선: 수안 누락, 타 노선 4개 역 삽입, 구 역명(동부산대) 사용
C5	동해선: 교대·신해운대·송정 누락, 민락·수영 잘못 삽입, 오타(헬내→월내)
C6	경전철: 오타(괴법→괘법르네시떼)

추가 지적 2건: ① CSV→데이터 생성 스크립트 부재(수정 시 640줄 수작업 위험), ② bfs() 는 정확히는 0-1 BFS이므로 보고서에 명시 필요.

특히 README의 "1·4호선 중복 등재" 참고사항은 데이터 오류를 정상 동작으로 합리화한 것임이 확인되었다. v0 데이터가 AI 생성물이었던 점을 고려하면, AI 산출물을 검증 게이트 없이 수용할 경우 오류가 코드뿐 아니라 문서화 단계까지 그대로 통과한다는 것을 보여준 사례였다.

단계 3 — 수정 스펙 생성 (spec_1)

프롬프트 원문:

"그럼 수정 사항들을 정리해서 SDD로 만들어줄래?"

AI 출력 요약: spec_1 생성 — 변경 요약 C1~C9, 노선별 정보 역 목록(공식 데이터 대조 전제 명시), 환승역 테이블 v1(거제 3↔동해선 추가), build_graph.py 신규 스펙(생성 규칙 4 + 내장 검증 4), 검증 게이트 V1~V7, 결정 사항 D1/D2.

단계 4 — 수정 실행 및 검증

spec_1을 입력으로 CSV 수정, build_graph.py 작성, metro_data.py 재생성을 수행했다. 알고리즘 코드(busan_subway.py)는 diff 0줄로 무변경을 유지했다(스펙의 "코드 변경 없음" 제약 준수 확인).

2-3. 본인 수정 사항 (사람의 결정·개입)

AI 출력을 그대로 수용하지 않고 다음을 직접 결정·수행했다.

1. **검증 게이트 채택** — v0의 근본 원인을 "데이터 검증 단계 부재"로 진단하고, 알려진 경로 기반 검증(V2~V7)과 데이터 대조(V1)를 워크플로의 필수 게이트로 추가했다.
2. **D1 결정: 동해선 동래역 분리** — v0은 1·4호선 동래와 동해선 동래를 한 노드로 병합했으나, 공식 환승역이 아니므로(간접환승) 별도 노드 동래(동해선)로 분리했다. 데이터 출처와의 일치를 사용 편의보다 우선한 판단이다.
3. **D2 결정: 역간 2분 균일 유지** — v1 범위를 데이터 정합성으로 한정하고, 실측 시간 반영은 향후 개선 사항으로 분류했다.
4. **정본 역 목록 재검증** — AI가 제시한 수정안 역 목록 자체도 검증 대상으로 취급했다. AI의 외부 검색 교차 확인(4호선)에 더해, 사람이 정답을 아는 경로 기반 검증(V2~V7)으로 전 노선의 정합성을 표본 확인하여 확정했다.
5. **작업 분할** — 토큰·컨텍스트 한도를 고려해 리뷰 / 스펙 / 실행 / 검증을 별도 세션으로 분리해 진행했다.

2-4. 반영 여부

항목	반영	확인 근거
C1~C6 데이터 수정	✓	노선별 역 수 40/43/17/14/21/23 일치, 오류 지점 전수 확인
C7 환승 테이블 (거제 추가 등)	✓	병합 환승역 12개, V7 통과
C8 build_graph.py	✓	내장 검증 4종 통과, 총 노드 146 = 기대값
C9 README 정정	✓	오류 합리화 문구 삭제, 동명이역 목록 축소(좌천·동래)
0-1 BFS 명시	✓	본 보고서 3장에 기술
D1 (동래 분리)	✓	V5 경로에 동래(동해선) 노드 확인
D2 (2분 균일 유지)	✓	가중치 무변경
알고리즘 무변경 제약	✓	busan_subway.py diff 0줄
검증 게이트 V2~V7	✓ 전 항목 통과	V2: 1호선 직통 24역 / V3: 1정거장 2분 / V4: 4호선 직통 14역 26분 (실측 25분 근사) / V5: 동해선 직통 23역 / V6: 오염 역 미출현 / V7: 거제 환승 동작

3. 적용한 자료 구조 및 알고리즘 — 요약 및 설명

3-1. 요약

구분	선택	목적
자료 구조	인접 리스트 (dict + list of tuples)	희소 그래프인 지하철망의 공간·탐색 효율
알고리즘 ①	0-1 BFS (collections.deque)	최소 환승 경로
알고리즘 ②	다익스트라 (heapq 우선순위 큐)	최단 시간 경로
보조 구조	(역, 노선) 상태 튜플 + dict 방문 체크	환승을 탐색 상태에 반영

같은 출발·도착에 대해 두 알고리즘을 동시에 실행하여 "최소 환승 경로"와 "최단 시간 경로"가 다를 수 있음을 비교 출력하는 것이 프로그램의 핵심이다.

3-2. 그래프 모델링

부산 도시철도 전 노선(1~4호선, 부산김해경전철, 동해선)을 다음과 같이 그래프로 표현했다.

```
GRAPH = {
    "서면": [("범내골", 2, "1호선"), ("부전", 2, "1호선"),
             ("전포", 2, "2호선"), ("부암", 2, "2호선")],
    ...
}
```

- **노드 = 역** (총 146개). 환승역(서면·연산 등 12개)은 노선별로 따로 두지 않고 **하나의 노드로 병합**했다.
- **에지 = (이웃역, 소요시간 2분, 노선명)**, 양방향 (총 152개).
- **환승 처리**: 별도의 "환승 에지"를 두지 않고, 에지에 붙은 노선 라벨을 이용해 **탐색 중 직전 주행 노선과 다른 노선의 에지로 이동하면 환승으로 간주**하고 3분을 가산한다. 환승이 데이터가 아니라 탐색 상태의 문제가 되므로, 데이터 구조가 단순해지고 두 알고리즘이 같은 그래프를 공유할 수 있다.
- **동명이역 처리**: 이름은 같지만 물리적으로 다른 역(좌천, 동래의 동해선 역)은 좌천(동해선)처럼 접미사를 붙여 별도 노드로 구분했다.

데이터 파이프라인은 busan_metro.csv (단일 진실 공급원) → build_graph.py (변환 + 검증) → metro_data.py (자동 생성된 GRAPH) 구조로, 실행 프로그램은 생성된 인접 리스트만 import한다.

3-3. 자료 구조: 왜 인접 리스트인가

그래프 표현 3가지 방식을 비교해 선택했다.

방식	공간	이웃 탐색	본 과제 적용 시
인접 행렬	$O(N^2)$	$O(N)$ 행 순회, 연결 확인은 $O(1)$	$146 \times 146 = 21,316$ 칸 중 대부분이 0
인접 리스트	$O(N + E)$	$O(\text{차수})$	노드 146 + 방향 에지 304개만 저장
엣지 리스트	$O(E)$	$O(E)$ 전체 순회	이웃 탐색마다 304개 전수 순회

지하철망은 역당 연결이 2~4개에 불과한(최대 차수 4: 연산역) **희소 그래프**다. 인접 행렬은 21,316칸 중 304칸만 의미를 갖게 되어 약 98.6%가 낭비되고, 엣지 리스트는 BFS·다익스트라가 매 단계 요구하는 "현재 역의 이웃 조회"가 $O(E)$ 로 비효율적이다. 인접 리스트는 공간을 실제 연결만큼만 쓰면서 이웃 조회가 $O(\text{차수})$ 로 끝나므로, 두 탐색 알고리즘 모두에 가장 적합하다. Python에서는 dict (역이름 → 에지 리스트)로 구현해 역 조회도 평균 $O(1)$ 이다.

3-4. 탐색 상태 설계: (역, 노선)

두 알고리즘의 공통 핵심은 방문 상태를 역 단위가 아니라 **(역, 노선)** 쌍 단위로 관리한 것이다. 같은 역이라도 "1호선을 타고 도착한 서면"과 "2호선을 타고 도착한 서면"은 이후 비용(환승 여부)이 달라지므로 서로 다른 상태다. 역 단위로만 방문 체크를 하면 먼저 도착한 노선이 이후의 더 좋은 경로를 차단하는 오답이 발생한다. 상태 수는 158개((역, 노선) 조합)로, 노드 수 대비 부담은 미미하다.

3-5. 알고리즘 ①: 0-1 BFS — 최소 환승 경로

일반 BFS로는 부족한 이유. BFS는 모든 에지의 비용이 같을 때 최단을 보장한다. 그러나 이 문제의 목적 함수는 "환승 횟수"이고, 에지 비용은 같은 노선 이동 = 환승 0회, 노선 변경 = 환승 1회로 **0과 1 두 가지**다. 이런 0/1 가중치 그래프에는 BFS의 확장형인 **0-1 BFS**가 정확히 들어맞는다.

동작 원리. 큐 대신 양방향 큐(deque)를 사용한다.

- 같은 노선으로 이동(비용 0) → `appendleft()` — 큐 **앞**에 넣어 먼저 처리
- 노선이 바뀌는 이동(비용 1) → `append()` — 큐 **뒤**로 보냄

이렇게 하면 deque 안의 상태들이 항상 환승 횟수 오름차순을 유지하여, 다익스트라의 우선순위 큐 없이도 $O(V + E)$ 에 최소 환승을 보장한다. 동물(같은 환승 횟수) 시에는 소요 시간이 적은 경로를 선택하도록 (환승, 시간) 사전순 비교를 사용했다.

상태: (환승횟수, 소요시간, 현재역, 현재노선, 경로)
방문 체크: $best[(역, 노선)] = (환승, 시간) \text{ 최소값}$

3-6. 알고리즘 ②: 다익스트라 — 최단 시간 경로

선택 이유. 시간 가중치는 이동 2분 + 환승 시 3분 가산으로 예지마다 비용이 달라지므로(2 또는 5), BFS로는 최단 시간을 보장할 수 없고 음수 가중치가 없으므로 다익스트라가 표준 해법이다.

동작 원리. heapq 최소 힙을 우선순위 큐로 사용한다. 힙에서 꺼내는 순간 그 상태까지의 최단 시간이 확정된다는 탐욕적(greedy) 성질에 따라, 도착역을 처음 꺼내는 순간 즉시 반환한다.

힙 상태: (소요시간, 환승횟수, 현재역, 현재노선, 경로) ← 시간 기준 최소 힙
 $dist[(역, 노선)] = \text{현재까지의 최단 시간 (더 느린 중복 상태는 스킵)}$

힙 튜플의 두 번째 원소를 환승 횟수로 두어, 시간이 같다면 환승이 적은 경로가 먼저 확정되도록 했다.

복잡도. 상태 그래프 기준 $O(E \log V) - V = (\text{역, 노선})$ 상태 158개, $E = \text{방향 에지}$ 304개 수준으로 즉시 응답한다.

3-7. 두 알고리즘 비교

	0-1 BFS	다익스트라
목적 함수	환승 횟수 (시간은 동물 처리)	소요 시간 (환승은 동물 처리)
자료 구조	deque	heapq (최소 힙)
시간 복잡도	$O(V + E)$	$O(E \log V)$
보장	0/1 가중치에서 최단	음수 없는 임의 가중치에서 최단

두 결과는 자주 일치하지만, "한 번 갈아타면 빠른 노선이 있는" 구간에서 갈라진다. 예컨대 환승 페널티(3분)를 감수하더라도 경유 역 수가 크게 줄어드는 경로가 있으면 다익스트라는 환승을 선택하고, 0-1 BFS는 직통을 고수한다. 동일 그래프·동일 환승 모델 위에서 목적 함수만 바꿔 비교한 것이므로, 차이는 순수하게 알고리즘의 최적화 기준에서 나온다.

4. 구현 및 보고서 작성 시 어려웠던 점 및 향후 개선 사항

4-1. 구현 시 어려웠던 점

① 데이터를 인접 리스트로 변환하고 "확인"하는 것 (가장 어려웠던 부분)

CSV 형태의 역 목록을 인접 리스트로 변환하는 코드 자체는 어렵지 않았다. 정말 어려웠던 것은 **변환된 데이터가 실제 노선과 일치하는지 확인하는 일**이었다. 노드 146개, 방향 에지 304개를 눈으로 전수 검수하는 것은 사실상 불가능했고, 프로그램은 데이터가 틀려도 "그 틀린 그래프 안에서의 정답"을 아무 오류 없이 출력하기 때문에 실행 결과만으로는 데이터 오류를 감지할 수 없었다.

실제로 초기 데이터(v0)에는 역 삽입·누락·순서 오류·오타가 4개 노선에 걸쳐 존재했고(예: 4호선 전용역 5개가 1호선에 삽입, 2호선 광안역 누락), 탐색 알고리즘은 정상 동작하면서도 현실과 다른 경로를 출력하고 있었다. 이를 통해 얻은 결론은 "데이터 정합성은 코드 정합성과 별개의 검증 축"이라는 것이다.

해결 방법 — 검증의 자동화 + 게이트화:

1. `build_graph.py` 에 검증을 내장 — 노선별 역 수 대조, 에지 대칭성($A \rightarrow B$ 가 있으면 $B \rightarrow A$ 존재) 검사, 동명이역 충돌 검출, 총 노드 수(146) 확인. 하나라도 어긋나면 생성 자체를 실패시킨다.
2. 알려진 정답 경로 기반 검증 게이트(V1~V7) — "남포→노포는 1호선 직통 24역", "미남→안평은 4호선 직통 26분(실제 25분 근사)"처럼 사람이 정답을 아는 경로로 그래프 전체의 정합성을 표본 검사했다.

② 환승 모델링과 (역, 노선) 상태

환승을 그래프에 어떻게 표현할지가 두 번째 난관이었다. 환승역을 노선별로 분리하고 환승 에지를 추가하는 방식은 데이터가 복잡해지고, 단일 노드로 병합하면 "방금 어떤 노선을 타고 왔는지"를 그래프가 기억하지 못한다. 결국 방문 상태를 역 단위가 아니라 (역, 노선) 쌍으로 관리하는 설계로 해결했는데, 역 단위 방문 체크가 왜 오답을 만드는지(먼저 도착한 노선이 더 좋은 경로를 차단) 이해하는 데 시간이 걸렸다.

③ 동명이역 처리

이름은 같지만 물리적으로 다른 역(1호선 좌천 vs 동해선 좌천)이 존재해, 이름을 그대로 노드 키로 쓰면 서로 다른 역이 하나로 합쳐지는 문제가 있었다. 공식 환승역만 병합하고 나머지 이름 충돌은 좌천(동해선) 처럼 접미사로 분리하는 규칙을 세워 해결했다.

4-2. 보고서 작성 시 어려웠던 점

① "우선순위 큐"와 "다익스트라"의 관계를 정확히 서술하기

처음에는 우선순위 큐(Priority Queue)와 다익스트라를 별개의 선택지처럼 생각해 "어느 쪽을 채택할지"를 고민했다. 그러나 정리 과정에서 둘은 비교 대상이 아니라 **층위가 다른 개념**임을 명확히 하게 되었다. 우선순위 큐는 자료 구조이고, 다익스트라는 그 자료 구조를 핵심 부품으로 사용하는 알고리즘이다.

즉 본 과제는 "우선순위 큐 대신 다익스트라를 채택"한 것이 아니라, **최단 시간 문제의 해법으로 다익스트라 알고리즘을 채택하고, 그 구현 수단으로 우선순위 큐(heapq 최소 힙)를 사용한 것**이다. 우선순위 큐가 없으면 매 단계 "현재까지 시간이 가장 짧은 상태"를 찾는 데 $O(V)$ 선형 탐색이 필요하지만, 최소 힙은 이를 $O(\log V)$ 로 줄여준다 — 다익스트라의 효율이 우선순위 큐라는 자료 구조에서 나온다는 점이 자료구조 과목 관점에서의 핵심이었다. 이 관계를 스스로 설명할 수 있게 정리한 것이 보고서 작성 과정의 가장 큰 학습이었다.

② 수업 범위와 구현 사이의 간극 설명

최소 환승 탐색에 사용한 0-1 BFS는 수업에서 배운 일반 BFS의 확장형이다. "왜 일반 BFS로는 부족한가(에지 비용이 0과 1로 갈라지기 때문)"에서 출발해, deque의 앞/뒤 삽입으로 정렬 효과를 내는 변형임을 수업에서 배운 BFS의 언어로 풀어 쓰는 데 공을 들였다.

4-3. 설계 의도 회고: 왜 객체지향인가

객체지향으로 작성한 가장 큰 이유는 **다른 노선도(다른 도시)도 포함할 수 있는 구조**를 만들기 위해서다. BusanMetro 클래스는 그래프를 생성자 주입으로 받기 때문에(`BusanMetro(graph)`), 서울 지하철 등 다른 도시의 인접 리스트를 만들어 주입하면 탐색 알고리즘과 출력 코드는 한 줄도 수정하지 않고 재사용할 수 있다.

데이터(GRAPH) ←교체 가능→ 알고리즘(BusanMetro) ←분리→ 입출력(MetroApp)

- `RouteResult` (값 객체): 탐색 결과를 표준화 — 알고리즘이 늘어나도 출력 코드는 동일
- `BusanMetro` : 그래프 보관 + 탐색 알고리즘 캡슐화 — 데이터 출처와 무관
- `MetroApp` : 출력·입력 루프 — 탐색 로직과 무관

이 분리 덕분에 실제로 v1 데이터 수정 작업에서 데이터만 전면 교체하면서 알고리즘 코드는 diff 0줄로 유지할 수 있었다 — 확장성을 위한 설계가 유지보수에서 먼저 효과를 본 사례다.

4-4. 향후 개선 사항

1. **실측 소요 시간 가중치 도입** — 현재는 역간 2분 균일 가중치라 최단 시간 탐색이 역 수 최소화와 유사해진다. 공공데이터(`data.go.kr`)의 실측 역간 소요 시간을 적용하면 다익스트라와 0-1 BFS의 결과 차이가 더 뚜렷해져, 두 알고리즘 비교의 의미가 강해진다.
2. **다른 도시 노선도 확장** — 4-3의 구조를 활용해 서울 지하철 CSV를 추가하고, 실행 시 도시를 선택하는 기능. 그래프 주입 설계의 실증이 된다.
3. **간접환승 모델링** — 동해선 동래역처럼 공식 환승역은 아니지만 도보 이동이 가능한 역 쌍을 별도 가중치의 "도보 예지"로 추가하면 더 현실적인 경로가 나온다.
4. **검증 게이트의 테스트 자동화** — 현재 수동으로 확인하는 V2~V7 정답 경로 검증을 `pytest` 단위 테스트로 전환해, 데이터 수정 시마다 자동 회귀 검사가 돌게 한다.
5. **경로 복원 최적화** — 현재는 탐색 상태마다 경로 리스트를 복사해 들고 다닌다. 역 수가 많은 도시로 확장할 경우 직전 상태 포인터(`prev`) 방식으로 바꿔 메모리를 줄일 수 있다.